ServiceLink: Project Planning & Technical Design Document

Nov 5, 2026 · @Kinza

1. Project overview

ServiceLink is a MERN-stack marketplace where customers book verified local service providers (tutors, electricians, cleaners, tailors) into conflict-free time slots. It was chosen because it covers every Month 3 requirement and grows naturally into a final year project.

Problem statement. Finding a reliable local service provider still runs on word of mouth and WhatsApp groups. Customers cannot see verified profiles, real availability, or past reviews. Providers lose jobs to double scheduling and have no booking history or reputation record.

Objectives

- Let customers search, compare, and book providers by category, city, price, and rating.
- Let providers publish services and weekly availability, and accept or reject bookings.
- Prevent double booking with server-side slot checks.
- Let admins approve providers, manage categories, and monitor all bookings.
- Allow reviews only for completed bookings, with ratings that update automatically.
- Ship a deployed, documented, tested portfolio project by the end of Week 4.

Why this project suits as a portfolio

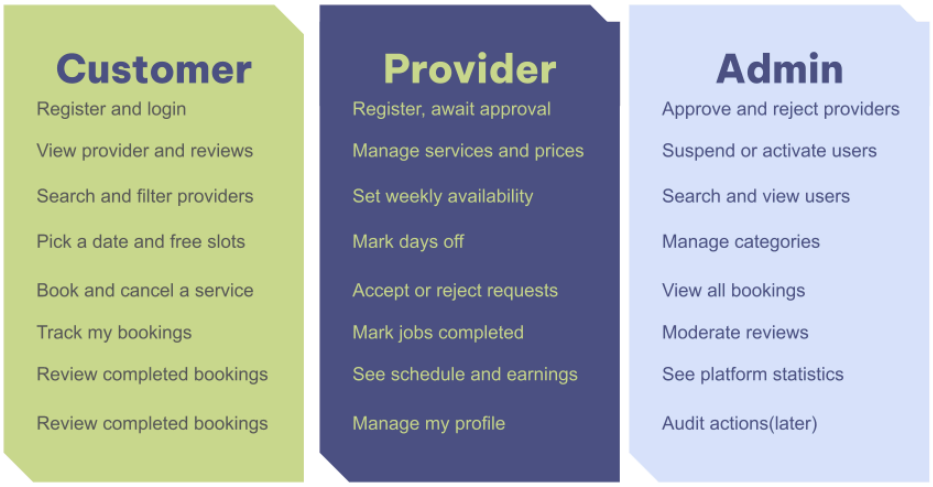
- Three roles, seven collections, and real business rules, so it is clearly not a basic CRUD form (Month 3 requirement).
- The hard parts are the ones interviewers ask about: availability logic, booking state machine, role-based access, aggregation.
- Clean extension paths for phases: online payments (Stripe or JazzCash), map-based search, real-time notifications, and a recommendation engine.

2. Target users and roles

Three roles share one login system; the backend decides what each can access based on the role stored in their JWT.

Role	Who they are	Can do	Cannot do
Customer	Anyone needing a service at home or locally	Browse and search providers, book a slot, cancel, review completed jobs, see own bookings	Edit services, see other users' bookings, access admin pages
Provider	Freelancers or small businesses offering services	Create services, set weekly availability, accept or reject bookings, mark jobs completed, view own earnings summary	Book as a provider for others, approve themselves, edit reviews
Admin	Platform operator (seeded account)	Approve or suspend providers, manage categories, view all bookings and users, remove abusive reviews	Book services or edit provider listings directly

Authentication answers who you are (register, login, JWT). **Authorization** answers what you may do (role checks plus ownership checks; for example, a provider can only change their own bookings).



The three roles barely overlap: only customers create bookings, only providers change their status, and only admins decide who may appear on the platform.

3. Features and functional requirements

Feature	Roles	Priority
Register, login, logout with JWT and bcrypt	All	Must
Role-based protected routes (API and React)	All	Must
Provider profile and admin approval	Provider, Admin	Must
Service listings (title, price, duration, category)	Provider	Must
Search and filter by category, city, price, rating	Customer	Must
Weekly availability and generated time slots	Provider	Must
Booking creation with double-booking prevention	Customer	Must
Booking status flow: pending, confirmed, rejected, completed, cancelled	Customer, Provider	Must
Reviews and ratings for completed bookings only	Customer	Must
Admin dashboard: users, providers, bookings, categories	Admin	Must
Customer and provider dashboards	Customer, Provider	Must
Input validation and consistent error responses	All	Must
Email notifications on booking events	All	Should
Provider earnings summary and charts	Provider	Should
Image upload for provider profile and services	Provider	Should
Online payment (Stripe or JazzCash), map search, real-time alerts	All	Could

Key functional rules

1. A slot can belong to at most one active booking; the server rechecks availability inside the booking request, not just in the UI.
2. Only the provider who owns a booking can confirm, reject, or complete it; only its customer can cancel it.
3. A review requires a completed booking, one review per booking.

4. A provider's average rating and review count update whenever a review is added or removed.
5. Suspended or unapproved providers never appear in search results.

3.1 Authentication and account (AUTH)

ID	Requirement	Roles	Priority
AUTH-1	Register with name, email, password, and role (customer or provider); email must be unique	All	Must
AUTH-2	Password needs 8 or more characters with a letter and a number; stored only as a bcrypt hash	All	Must
AUTH-3	Login with email and password returns a JWT and the user profile without the password	All	Must
AUTH-4	Logout clears the token on the client; protected pages redirect to login	All	Must
AUTH-5	The session survives a page refresh by restoring the user from the token (GET /auth/me)	All	Must
AUTH-6	Suspended accounts cannot log in and see a clear message	All	Must
AUTH-7	Edit own profile (name, phone, city) and change password by entering the current one	All	Should
AUTH-8	Reset a forgotten password through an emailed one-time link	All	Could
AUTH-9	Verify email address after signup	All	Could
AUTH-10	A seed script creates the admin account; nobody can register as admin	Admin	Must

3.2 Provider profile and onboarding (PRV)

ID	Requirement	Roles	Priority
PRV-1	Provider profile with bio, city, years of experience, categories, and photo	Provider	Must
PRV-2	New providers start as pending; pending providers can log in but are not listed or bookable	Provider, Admin	Must

PRV-3	Rejected providers see the reason and can edit and resubmit	Provider	Should
PRV-4	Public provider page shows profile, services, average rating, and reviews	Public	Must
PRV-5	Provider can pause new bookings (for example, while travelling) without deleting anything	Provider	Should

3.3 Services and categories (SRV)

ID	Requirement	Roles	Priority
SRV-1	Provider creates, edits, deactivates, and deletes own services (title, description, price, duration, category)	Provider	Must
SRV-2	A service with future pending or confirmed bookings cannot be deleted, only deactivated	Provider	Must
SRV-3	Price must be a positive whole number in PKR; duration must not exceed the provider's slot length	Provider	Must
SRV-4	Admin creates, renames, and deletes categories; a category in use cannot be deleted	Admin	Must
SRV-5	Up to three images per service	Provider	Should

3.4 Search and discovery (SRC)

ID	Requirement	Roles	Priority
SRC-1	Keyword search across service title and provider name	Public	Must
SRC-2	Filter by category, city, price range, and minimum rating	Public	Must
SRC-3	Sort by rating, price (low to high), and newest	Public	Must
SRC-4	Paginated results (10 per page) with total count	Public	Must
SRC-5	Only approved, active, and unpaused providers appear in results	Public	Must
SRC-6	Friendly empty state when nothing matches, with a button to clear filters	Public	Must
SRC-7	Nearby search on a map using provider coordinates	Public	Could

3.5 Availability and slots (AVL)

ID	Requirement	Roles	Priority
AVL-1	Provider sets working hours per weekday and a slot length (30 or 60 minutes)	Provider	Must
AVL-2	Provider marks specific dates as days off	Provider	Should
AVL-3	Free slots for a date = weekly rule minus days off minus active bookings minus past times	System	Must
AVL-4	Slots can be booked from today up to 30 days ahead	System	Must
AVL-5	Changing availability never cancels bookings that already exist	System	Must
AVL-6	A minimum notice period (default 2 hours) applies to same-day bookings	System	Should

3.6 Booking (BKG)

ID	Requirement	Roles	Priority
BKG-1	Customer picks service, date, and slot, adds address and notes, and submits; status starts as pending	Customer	Must
BKG-2	The server re-checks that the slot exists, is free, and is in the future; otherwise it returns 409	System	Must
BKG-3	Two simultaneous requests for one slot: exactly one succeeds (enforced by a database index)	System	Must
BKG-4	The price at booking time is saved on the booking so later price changes do not alter it	System	Must
BKG-5	Provider confirms or rejects a pending booking, with an optional reason	Provider	Must
BKG-6	Customer cancels a pending or confirmed booking; provider can cancel a confirmed one with a reason; the slot is freed	Customer, Provider	Must
BKG-7	Provider marks a confirmed booking completed only on or after its start time	Provider	Must

BKG-8	Only the status moves listed in section 11 are allowed; any other change returns 400	System	Must
BKG-9	Booking lists are filtered by status and date range and are paginated	Customer, Provider	Must
BKG-10	Pending bookings not answered within 24 hours expire automatically	System	Should
BKG-11	A customer cannot hold two overlapping bookings	Customer	Should
BKG-12	Customer reschedules a booking to another free slot	Customer	Could

3.7 Reviews (REV)

ID	Requirement	Roles	Priority
REV-1	One review per completed booking: rating 1 to 5 and a comment up to 500 characters	Customer	Must
REV-2	Only the customer of that booking can review it	Customer	Must
REV-3	Provider average rating and review count are recalculated when a review is added or removed	System	Must
REV-4	Admin removes abusive reviews	Admin	Must
REV-5	Provider posts one public reply to a review	Provider	Could
REV-6	Customer edits a review within 7 days	Customer	Could

3.8 Dashboards (DSH)

ID	Requirement	Roles	Priority
DSH-1	Customer dashboard: upcoming bookings, past bookings, and a rebook button	Customer	Must
DSH-2	Provider dashboard: today's schedule, pending requests, and counts by status	Provider	Must
DSH-3	Provider earnings summary from completed bookings	Provider	Should
DSH-4	Admin dashboard: totals for users, providers, and bookings by status, plus pending approvals	Admin	Must
DSH-5	Charts for bookings per week and revenue	Provider, Admin	Should

3.9 Administration (ADM)

ID	Requirement	Roles	Priority
ADM-1	Approve or reject provider applications	Admin	Must
ADM-2	Suspend or reactivate users; suspending cancels their future bookings	Admin	Must
ADM-3	View all bookings with filters by status, provider, and date	Admin	Must
ADM-4	Manage categories and moderate reviews	Admin	Must
ADM-5	Audit log of admin actions (who, what, when)	Admin	Could

3.10 Notifications (NTF) and system behaviour (SYS)

ID	Requirement	Roles	Priority
NTF-1	Email to the provider on a new booking and to the customer on confirm, reject, cancel, and complete	All	Should
NTF-2	In-app notification bell with unread count	All	Could
SYS-1	Every API response uses the shared success or error JSON shape	System	Must
SYS-2	Every write route validates input on the server; forms also validate in the browser	All	Must
SYS-3	Every screen shows loading, empty, error, and success states	All	Must
SYS-4	Friendly 403 and 404 pages; unknown IDs and malformed IDs return 404 or 400, never 500	All	Must
SYS-5	All list endpoints are paginated with a maximum page size of 50	System	Must
SYS-6	Health endpoint for the hosting platform to check	System	Must

4. Non-functional requirements

These describe how well the system must work. Each one has a way to check it, so it can be tested rather than just claimed. Performance numbers are targets for demo scale (about 1,000 providers and 10,000 bookings).

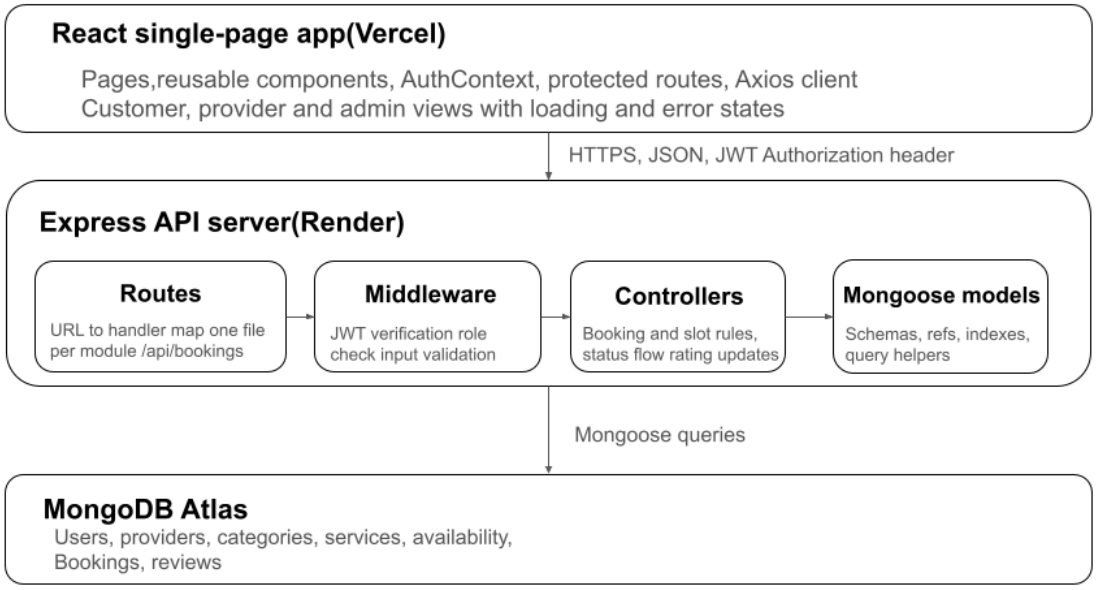
ID	Category	Requirement	How it is verified
NFR-1	Security	Passwords are hashed with bcrypt (cost 10 or more), never logged, and never returned by the API	Inspect the database and API responses
NFR-2	Security	JWT secret is a random string of 32 or more characters in an environment variable; tokens expire after 7 days	Check config; use an expired token in Postman
NFR-3	Security	Role and ownership checks run on the server for every protected route, not only in the UI	Call routes with another role's token and expect 403

NFR-4	Security	Request bodies are validated and sanitised: unknown fields dropped, strings trimmed, query operators such as $ne stripped	Send injection payloads and expect 400
NFR-5	Security	Security headers (helmet), CORS limited to the frontend origin, rate limit on login and register	Inspect response headers; repeat login 11 times quickly
NFR-6	Security	No secret is ever committed; production runs over HTTPS only	Search Git history; open the live site
NFR-7	Performance	Search and list endpoints answer in under 500 ms (95th percentile) at demo scale	Time requests in Postman or the browser network tab
NFR-8	Performance	Indexes cover search filters and booking lookups	Run explain() on the main queries
NFR-9	Performance	First page load under 3 seconds on broadband; routes are code-split; uploaded images are 200 KB or less	Browser Lighthouse report
NFR-10	Scalability	The server keeps no session in memory, so more instances can run behind a load balancer; database connections are pooled	Code review of the auth and DB setup
NFR-11	Reliability	Bad input or a failed database call returns a JSON error and never crashes the server; unhandled promise rejections are caught	Send malformed requests; stop the database and call the API
NFR-12	Reliability	A booking can never be double-booked, even under simultaneous requests	Fire two parallel booking requests in a test script
NFR-13	Data integrity	Money is stored as whole PKR amounts, dates in UTC and shown in Asia/Karachi time; deletes check references first	Review schemas; try deleting used records
NFR-14	Usability	Works from 360 px phone width upward; a booking takes no more than 5 steps from the home page	Manual test on phone and desktop

NFR-15	Usability	Forms show field-level errors; every action gives visible feedback (toast or inline message)	Manual walkthrough
NFR-16	Accessibility	Semantic HTML, labelled inputs, keyboard navigation, colour contrast of at least 4.5: 1, alt text on images	Keyboard-only test and Lighthouse accessibility score
NFR-17	Compatibility	Latest two versions of Chrome, Edge, Firefox and Safari, and mobile Chrome and Safari	Smoke test on each
NFR-18	Maintainability	Separate folders for routes, controllers, models, middleware, and services; ESLint and Prettier pass; no dead or debug code in the final commit	Lint run; code review
NFR-19	Testability	Postman collection covers every endpoint; slot generation and status transitions have automated unit tests	Run the collection and the test script
NFR-20	Observability	Requests are logged (Morgan); server errors are logged with a stack trace, but the stack is hidden from clients in production	Trigger an error and compare log and response
NFR-21	Portability	All configuration comes from environment variables; local setup works with npm install and npm run dev	Fresh clone on another machine
NFR-22	Documentation	README, API docs, diagrams, and setup steps stay in the repository and match the code	Mentor review checklist
NFR-23	Localisation	Prices in PKR, dates in Asia/Karachi time, English interface; text kept in one place so Urdu can be added later	Review UI strings
NFR-24	Privacy	Store only what is needed (name, email, phone, city, booking address); a user can see all their own data	Review schemas and profile page
NFR-25	Recoverability	Take a database export (mongodump) before the final demo and keep it outside the repository	Restore the dump into a test database

5. System architecture

A React single-page app talks to a stateless Express API over JSON, and only the API touches MongoDB.



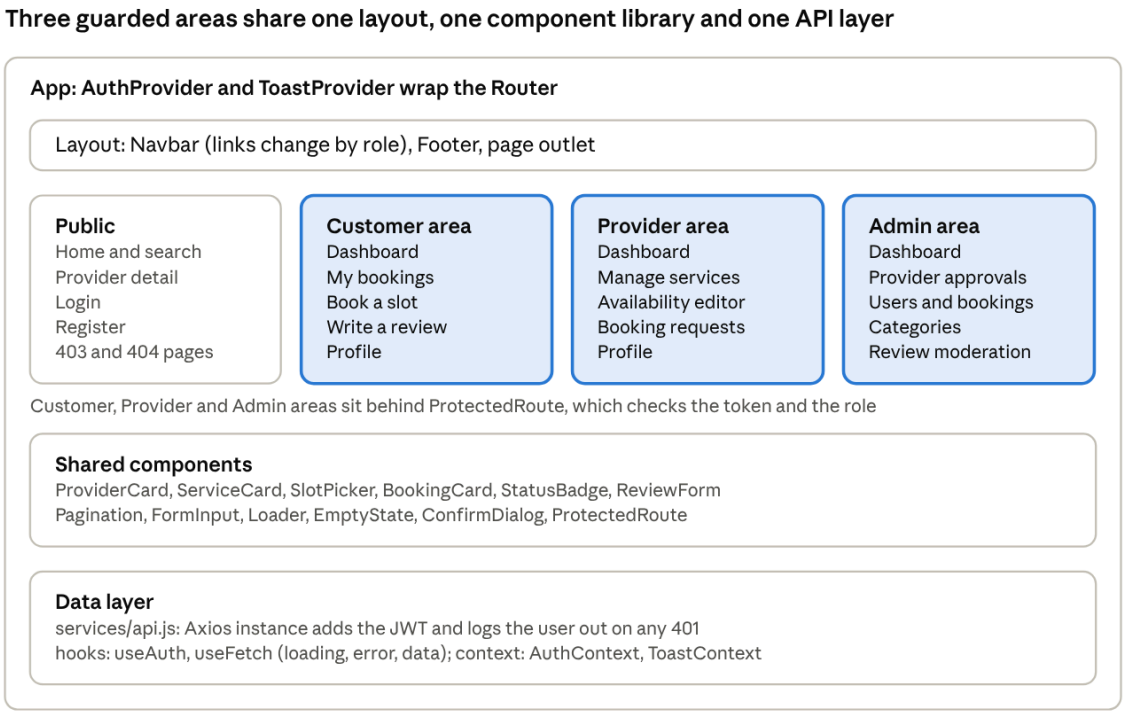
Each request travels through Routes, Middleware, Controllers, Models; the response returns along the same path.

Technical decisions

- **Separate frontend and backend** so each deploys on its own host and the API can later serve a mobile app.
- **JWT in the Authorization header** keeps the server stateless; the token carries user id and role.
- **References, not embedding,** for bookings, reviews, and services: they grow without bound and are queried on their own.
- **Slots are computed on the server** from the weekly schedule minus active bookings, so the UI can never book a taken slot.
- **One error-handling middleware** gives every failure the same JSON shape.

6. Frontend design

The React app is one layout, four route groups, and a shared set of small components, so every page is built from the same parts.



State management. No Redux is needed at this size.

- **`AuthContext`** holds the user, token, and loading flag and exposes **`login`**, **`register`**, and **`logout`** through the **`useAuth`** hook.
- Page data is fetched with a **`useFetch`** hook that returns **`data`**, **`loading`**, and **`error`**.
- Form fields live in component state; search filters live in the URL query string so results can be shared and survive a refresh.

Routing. React Router with nested routes. **`ProtectedRoute`** takes the allowed roles: no token sends the user to **`/login`** and back after login; a wrong role sends them to **`/403`**.

API layer. One Axios instance reads its base URL from **`VITE_API_BASE_URL`**. A request interceptor adds the Bearer token; a response interceptor logs the user out on 401. Each module has a small service file (**`authService`**, **`bookingService`**, and so on) so components never write URLs.

```
None
frontend/src/
  components/   shared UI (cards, slot picker, badges, dialogs)
  pages/        public/ customer/ provider/ admin/
  context/      AuthContext, ToastContext
  hooks/        useAuth, useFetch
  services/     api.js, authService.js, bookingService.js, ...
  routes/       AppRoutes.jsx, ProtectedRoute.jsx
  utils/        formatters, validators
  App.jsx
  main.jsx
```

7. Backend design

The Express app is split by responsibility: routes only map URLs, middleware guards and validates, controllers hold business rules, and models describe the data.

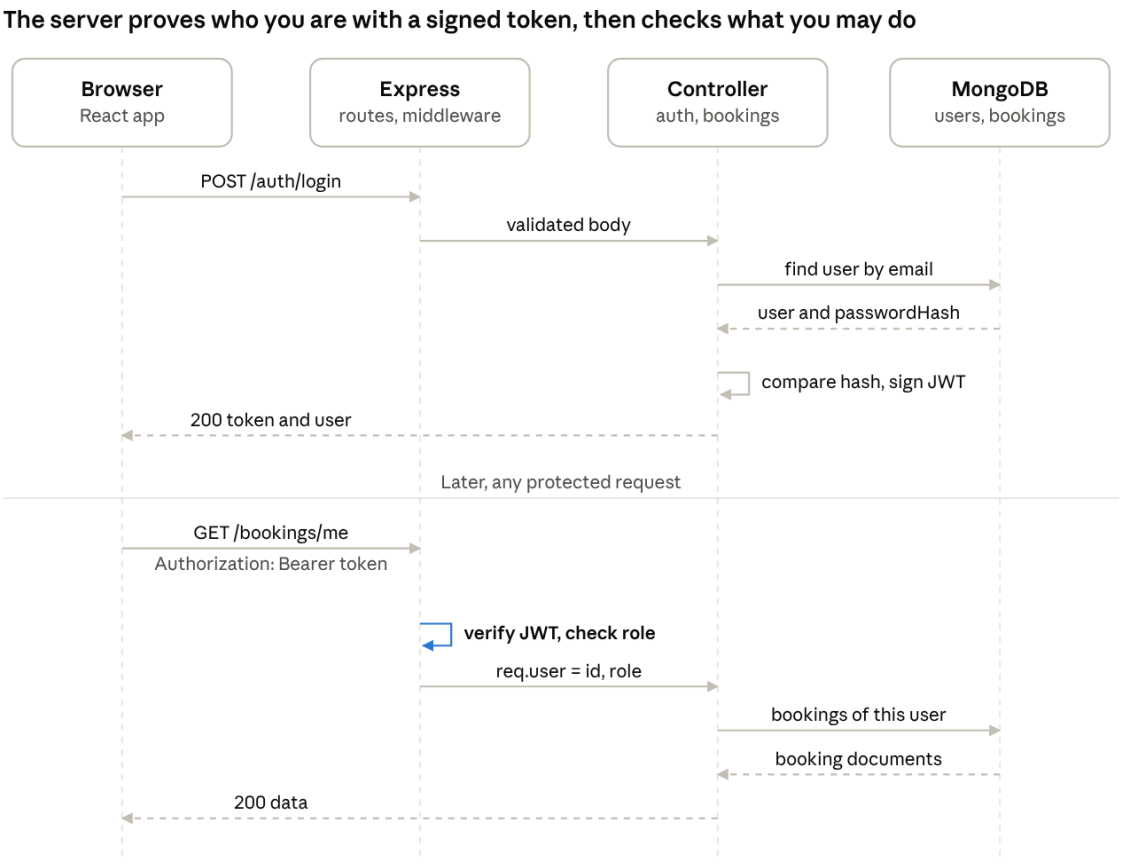
```
None
backend/
  src/
    config/       db.js (Mongoose connection), env.js
    models/       User, Provider, Category, Service, Availability,
Booking, Review
    routes/       one file per module
    controllers/  request handling and business rules
    services/     slotService, bookingService, ratingService,
emailService
    middleware/   authenticate, authorize, validate, notFound,
errorHandler
    utils/        AppError, asyncHandler, token helpers
    scripts/      seedAdmin.js, seedDemoData.js
    tests/        Jest tests for slots and status moves
    app.js        Express setup
    server.js     starts the server
  .env.example
```

Request pipeline, in order

1. **`helmet`**, **`cors`** (frontend origin only), **`express.json`** with a 10 KB limit
2. **`express-mongo-sanitize`** and **`morgan`** request logging
3. Rate limiter on the auth routes
4. Per route: **`authenticate`** (verify JWT), **`authorize(roles)`**, **`validate(rules)`**, then the controller
5. **`notFound`** for unknown URLs, then one **`errorHandler`** that turns any thrown **`AppError`** or database error into the shared JSON shape

Controllers are wrapped in **`asyncHandler`** so a rejected promise always reaches the error handler instead of crashing the process. Failures look like this:

```
JSON
{ "success": false, "message": "Validation failed", "errors": [ { "field": "email", "message": "Email is already registered" } ] }
```

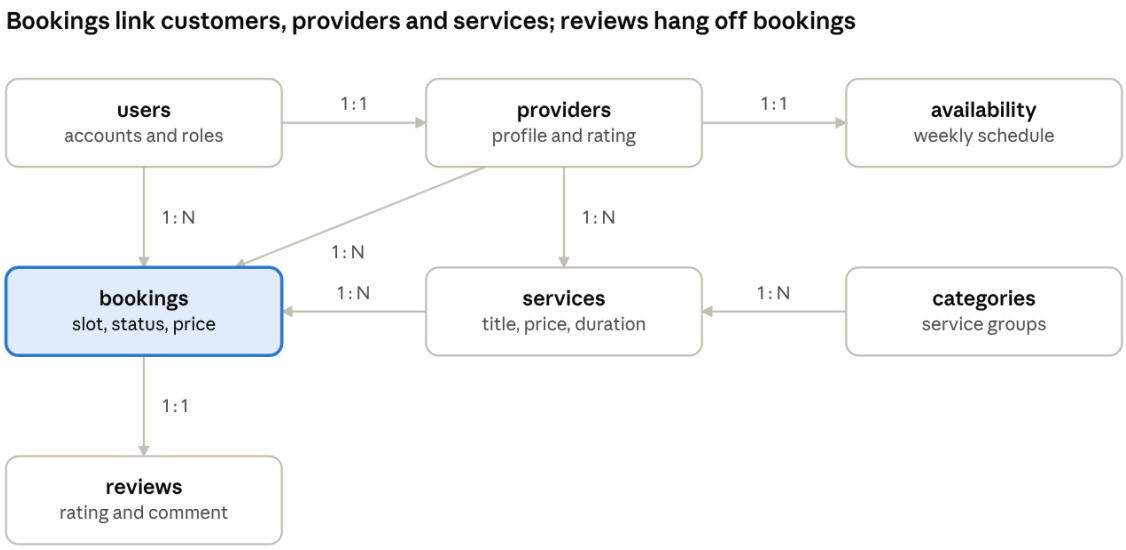


Missing or invalid token returns 401; a valid token with the wrong role returns 403.

The login half shows how a token is made; the lower half shows what every protected route repeats before any controller runs.

8. Database design

Seven MongoDB collections hold everything; bookings is the core one because it ties a customer, a provider, and a service to a single time slot.



Arrows point from the parent to the collection that stores its id. Reviews also store the customer and provider ids so ratings can be aggregated without extra lookups.

Collection	Key fields	Indexes and rules
users	name, email, passwordHash, role (customer, provider, admin), phone, city, isActive	Unique index on email; passwordHash is never returned by the API
providers	user (ref), bio, city, experienceYears, approvalStatus (pending, approved, rejected), avgRating, reviewCount	One per user; index on city, approvalStatus, avgRating
categories	name, slug, icon	Unique name

services	provider (ref), category (ref), title, description, price, durationMinutes, isActive	Index on category and price; index on provider
availability	provider (ref), weekly rules (dayOfWeek, startTime, endTime), slotMinutes, daysOff	One per provider
bookings	customer (ref), provider (ref), service (ref), date, startTime, endTime, status, address, notes, priceAtBooking	Unique index on provider, date, startTime for pending and confirmed bookings only, so a freed slot can be rebooked
reviews	booking (ref), customer (ref), provider (ref), rating 1 to 5, comment	Unique on booking; index on provider

Required fields, enums, and min/max limits are enforced in the Mongoose schemas and again by request validation.

9. API list

The API has 32 endpoints under **`/api`**, grouped by module. Every response uses one shape: **`{ success, message, data }`** on success and **`{ success: false, message, errors }`** on failure, with proper status codes (400, 401, 403, 404, 409, 500).

Module	Method and path	Access	Purpose
Auth	POST `/auth/register`	Public	Create customer or provider account
Auth	POST `/auth/login`	Public	Verify credentials, return JWT
Auth	GET `/auth/me`	Logged in	Current user from token
Categories	GET `/categories`	Public	List categories
Categories	POST `/categories`	Admin	Create category
Categories	PUT `/categories/:id`	Admin	Update category
Categories	DELETE `/categories/:id`	Admin	Delete unused category

Providers	GET `/providers`	Public	Search by category, city, price, rating, page
Providers	GET `/providers/:id`	Public	Provider profile with services
Providers	PUT `/providers/me`	Provider	Update own profile
Services	GET `/services`	Public	List and filter services
Services	GET `/services/:id`	Public	Service detail
Services	POST `/services`	Provider	Create service
Services	PUT `/services/:id`	Provider (owner)	Update service
Services	DELETE `/services/:id`	Provider (owner)	Remove service
Availability	GET `/availability/me`	Provider	Read own weekly schedule
Availability	PUT `/availability/me`	Provider	Save weekly schedule
Availability	GET `/providers/:id/slots?date=`	Public	Free slots for one day
Bookings	POST `/bookings`	Customer	Create booking, checks slot is free
Bookings	GET `/bookings/me`	Customer, Provider	Own bookings with status filter
Bookings	GET `/bookings/:id`	Participant, Admin	Booking detail
Bookings	PATCH `/bookings/:id/status`	Owner per rule	Confirm, reject, complete, or cancel
Reviews	POST `/bookings/:id/review`	Customer	Review a completed booking
Reviews	GET `/providers/:id/reviews`	Public	List provider reviews
Reviews	DELETE `/reviews/:id`	Admin	Remove abusive review

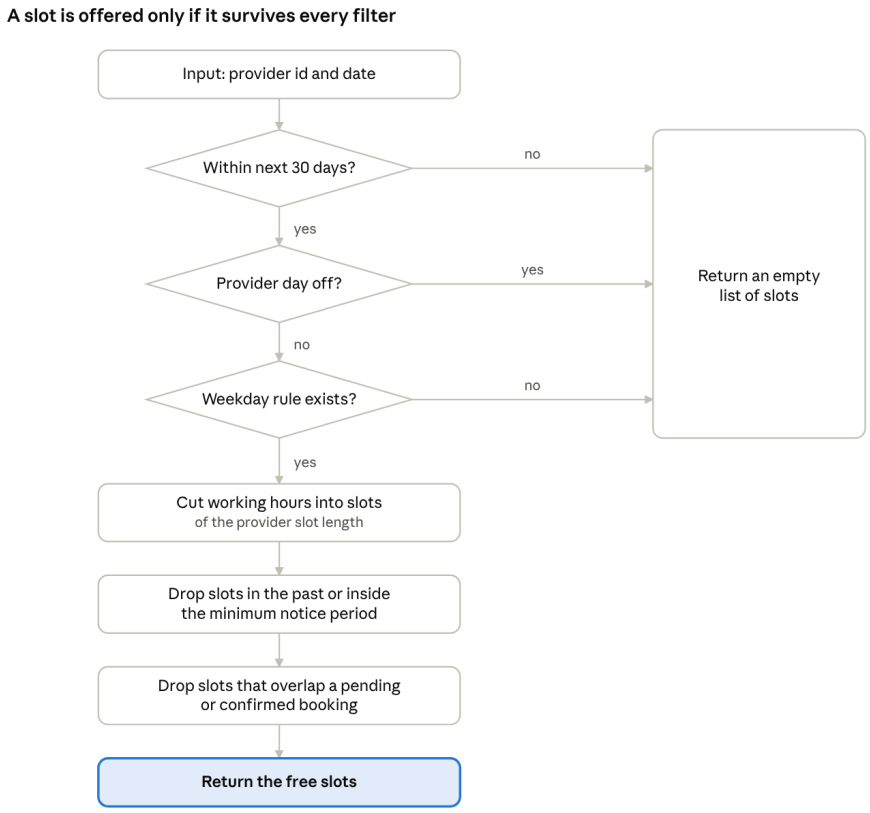
Admin	GET `/admin/stats`	Admin	Totals for dashboard
Admin	GET `/admin/users`	Admin	List users
Admin	PATCH `/admin/users/:id/status`	Admin	Activate or suspend user
Admin	GET `/admin/providers/pending`	Admin	Providers waiting for approval
Admin	PATCH `/admin/providers/:id/approval`	Admin	Approve or reject provider
Admin	GET `/admin/bookings`	Admin	All bookings with filters
Health	GET `/health`	Public	Deployment check

10. Key flows and business logic

Two pieces of logic carry most of the application: how free slots are calculated, and how a booking is created without double-booking.

10.1 Free-slot calculation

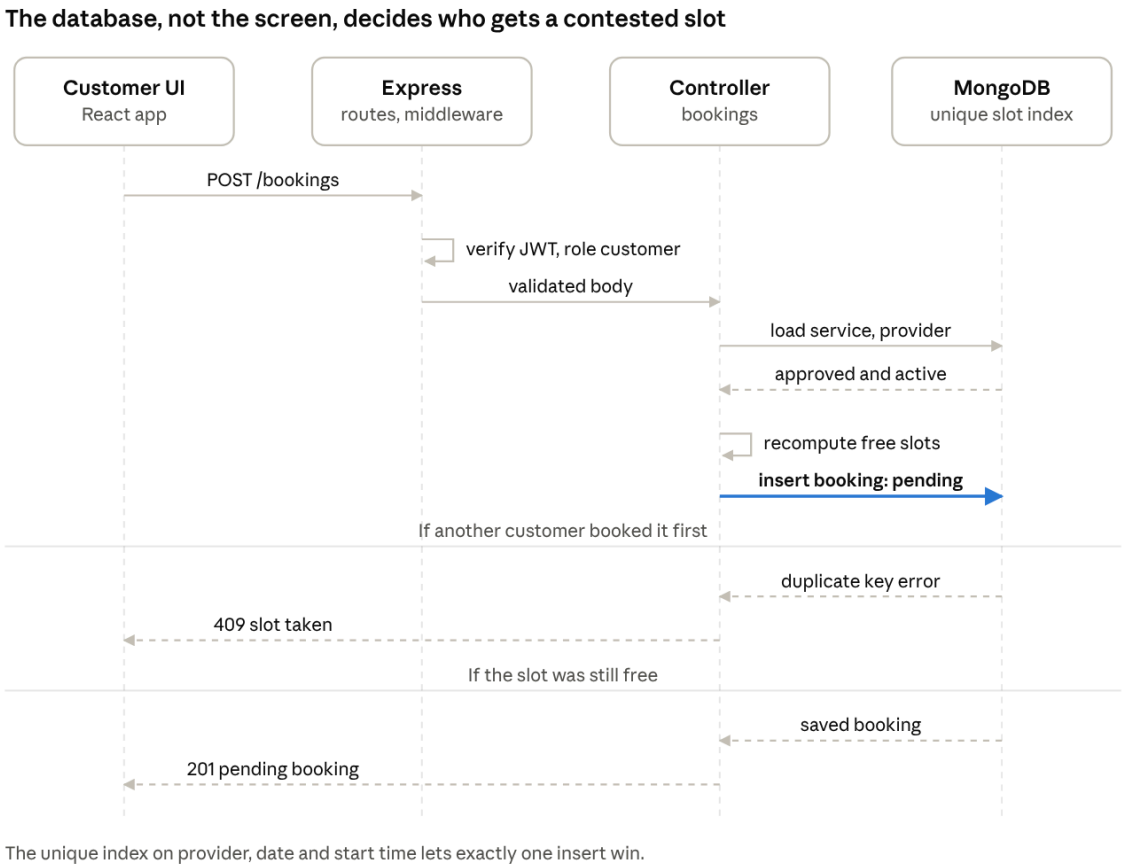
For a provider who works Monday 10:00 to 14:00 with 60-minute slots, a Monday request starts with four slots (10:00, 11:00, 12:00, 13:00); the filters then remove any that are past, too soon, or already taken.




```
None
function getFreeSlots(providerId, date):
  if date is outside today .. today + 30 days: return []
  if date is in provider.daysOff: return []
  rule = provider.weekly[weekday(date)]; if none: return []
  slots = split(rule.start, rule.end, provider.slotMinutes)
  slots = slots that start after now + minimumNotice
  taken = bookings of provider on date with status pending or
confirmed
  return slots that do not overlap any of taken
```

This function lives in **`slotService.js`** with no Express code inside it, so Jest can test it with plain inputs (cases: day off, no rule, past date, one taken slot, all slots taken).

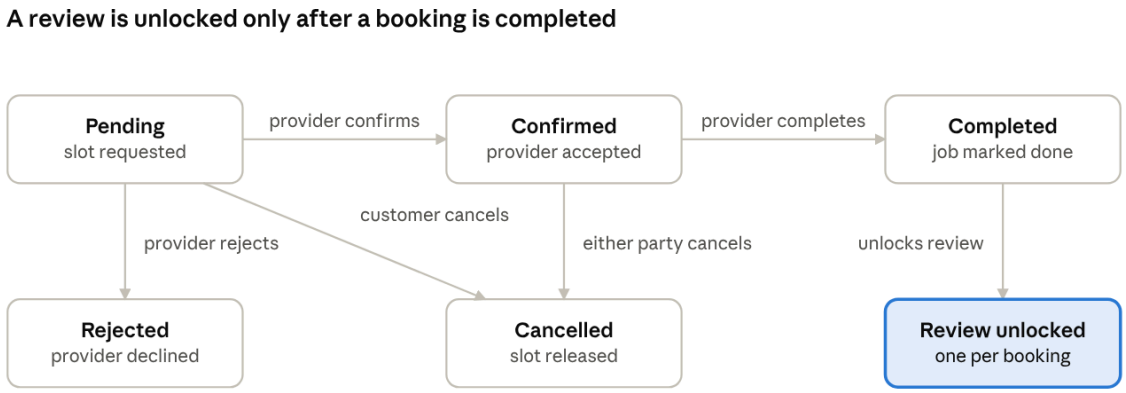
10.2 Creating a booking



- The screen shows slots, but the server recomputes them inside the booking request; a stale screen can never book a taken slot.
- The unique partial index is the final judge: when two customers submit at the same moment, the database accepts one insert. It rejects the other with a duplicate-key error, which the controller turns into a 409.
- On a 409, the UI refreshes the slot list and tells the customer to pick another time.
- The service price is copied into the booking, so later price edits do not change old bookings.
- The provider email (NTF-1) is sent after the booking is saved and never blocks the response.
- Each booking occupies exactly one slot; a provider chooses a slot length that fits their services.

11. User flow and pages

A booking moves through five statuses, and the status decides which button each role sees next.



Rejected and cancelled bookings free the slot immediately; only completed bookings can be reviewed.

Customer journey

1. Search or filter providers on the home page.
2. Open a provider, choose a service, date, and free slot.
3. Log in or register if not already signed in.
4. Confirm the booking and see it as pending.
5. Track it under My bookings; cancel while pending or confirmed.
6. Leave a rating and comment once the booking is completed.

Provider journey: register, wait for admin approval, add services, set weekly availability, accept or reject requests, mark jobs completed.

Page	Route	Access
Home and search	`/`	Public
Provider detail and slot picker	`/providers/:id`	Public, booking needs login
Login and register	`/login`, `/register`	Public

My bookings	`/bookings`	Customer, Provider
Customer dashboard	`/dashboard`	Customer
Provider dashboard: services, availability, requests	`/provider`	Provider
Admin dashboard: approvals, users, categories, bookings	`/admin`	Admin
Not found and unauthorized	`*`, `/403`	Public

12. Security design

Security is part of the Month 3 evaluation, so each likely threat is listed with the control that answers it. The JWT is kept in localStorage for simplicity; because that is exposed to script injection, the app never renders raw HTML and sets a content security policy. Moving the token to an httpOnly cookie is a planned FYP upgrade.

Threat	**How it could happen**	**Control**
Stolen passwords	Database leak	bcrypt hashes with salt; `passwordHash` excluded from queries by default
Brute-force login	Automated guessing	Rate limit on login and register; same message for wrong email and wrong password
Broken access control	A customer calls a provider or admin route, or edits another user's record by changing an ID	Role middleware plus an ownership check in every controller that changes data
Privilege escalation	Sending `role: admin` while registering	Register accepts only customer or provider; unknown fields are dropped; admin comes from a seed script
NoSQL injection	Sending `{ "$ne": null }` as an email or ID	express-validator type checks, express-mongo-sanitize, strict Mongoose schemas
Token theft through XSS	Malicious script reads localStorage	React escapes output, no raw HTML, helmet content security policy, 7-day expiry

CSRF	Another site triggers requests as the user	Token sent in the Authorization header, not a cookie, so browsers do not attach it automatically; CORS allows only the frontend origin
Double booking	Two requests arrive for the same slot	Unique partial index on provider, date, and start time for active bookings
Data exposure in errors	Stack traces returned to the client	Central error handler returns a short message in production and logs details on the server
Oversized or malformed requests	Huge JSON body, broken IDs	Body size limit of 10 KB, ObjectId validation on every `: id` parameter
Leaked secrets	`.env` pushed to GitHub	`.gitignore` from the first commit, `.env.example` with names only, secrets set in Render and Vercel dashboards

13. Testing plan

Testing happens at four levels: unit tests for the slot and status logic (Jest), API tests for every endpoint (Postman collection, saved in `docs/`), manual UI tests on desktop and a 360 px phone, and a final smoke test on the deployed URLs. The cases below are the minimum evidence for the Week 3 and Week 4 submissions; record each result with a screenshot or Postman run.

ID	Area	Test	Expected result	Covers
TC-01	Auth	Register a valid customer	201, token and user returned, no password in response	AUTH-1, AUTH-2
TC-02	Auth	Register with an email that exists	409 with a clear message	AUTH-1
TC-03	Auth	Register with a weak password	400 with a field error	AUTH-2
TC-04	Auth	Register with role admin	Rejected; account is not admin	AUTH-10
TC-05	Auth	Log in with correct details	200 and a token	AUTH-3

TC-06	Auth	Login with wrong password	401, same message as unknown email	AUTH-3
TC-07	Auth	Login as a suspended user	403 with account-suspended message	AUTH-6
TC-08	Access	Call a protected route without a token	401	NFR-3
TC-09	Access	Use a customer token on an admin route	403	NFR-3
TC-10	Access	Provider edits another provider's service	403	SRV-1
TC-11	Services	Create a service with a negative price	400	SRV-3
TC-12	Services	Delete a service that has a future booking	409, service stays	SRV-2
TC-13	Search	Filter by category, city, and minimum rating	Only approved, matching providers appear	SRC-2, SRC-5
TC-14	Search	Request page 2 of the results	Correct items and total count	SRC-4
TC-15	Slots	Ask for slots on a day off	Empty list	AVL-3
TC-16	Slots	Ask for a past date and a date 40 days ahead	Empty list or 400	AVL-4
TC-17	Booking	Book a free slot	201, status pending	BKG-1
TC-18	Booking	Send two booking requests for one slot at the same time	One 201 and one 409	BKG-2, BKG-3
TC-19	Booking	Complete a booking before its start time	400	BKG-7
TC-20	Booking	Customer cancels a confirmed booking	Status cancelled, slot free again	BKG-6
TC-21	Booking	Move a completed booking back to pending	400	BKG-8

TC-22	Reviews	Review a booking that is not completed	Rejected	REV-1
TC-23	Reviews	Review the same booking twice	409	REV-1
TC-24	Reviews	Add a 4-star review to a provider with one 5-star review	Average becomes 4.5, count 2	REV-3
TC-25	Robustness	Call a route with a malformed ID	400, not 500	SYS-4
TC-26	Admin	Suspend a user who has a future booking	Login blocked, booking cancelled	ADM-2
TC-27	UI	Complete a booking on a 360 px wide screen	Every step usable without sideways scrolling	NFR-14
TC-28	Deployment	Register, book, confirm, and review on the live URLs	Whole flow works against Atlas	NFR-21

14. Technology stack and GitHub repository

The stack is the MERN stack from the roadmap, with a few small libraries chosen because they are widely used in job postings.

Layer	Choice	Why
Frontend	React with Vite, React Router, Axios	Fast setup, component-based, matches Month 2
State	Context API plus a custom `useAuth` hook	Enough for auth and user state without extra libraries
Styling	Tailwind CSS	Quick responsive layouts, consistent look
Backend	Node.js, Express.js	Required by the roadmap
Database	MongoDB Atlas with Mongoose	Required; Atlas is also the deployment target

Auth	JWT, bcryptjs	Required; stateless and simple to protect routes
Validation	express-validator on the API, inline checks in forms	Clear validation errors on every write route
Testing	Postman collection; Jest with Supertest for key routes	Postman is required; Jest is a bonus
Deployment	Vercel (frontend), Render (backend), Atlas (database)	Free tiers; confirm AUREX's approved platforms

Repository layout

```
None
servicelink/
  frontend/        React app (components, pages, context, services)
  backend/         Express app (config, models, controllers,
routes, middleware)
  docs/            diagrams, Postman collection
  README.md
  .gitignore       node_modules, .env, build folders
```

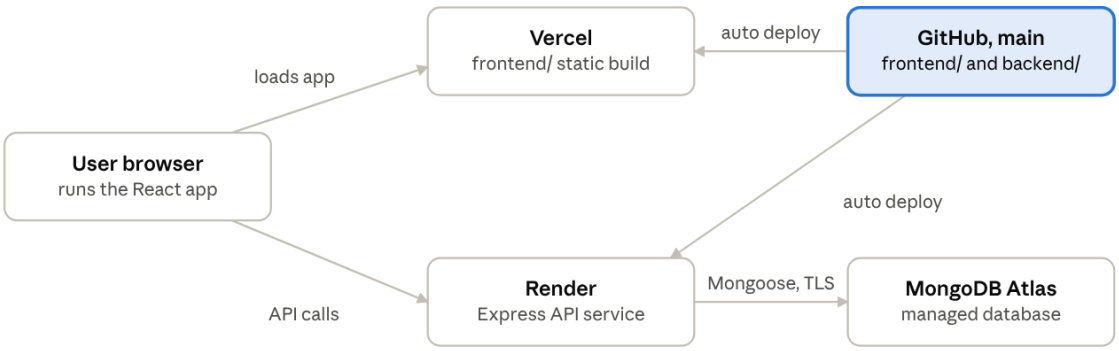
Git workflow

- **`main`** always runs; **`dev`** collects work; each feature gets a **`feature/<name>`** branch merged by pull request.
- Commit messages start with a verb and one change, for example, **`Add booking model with status enum`**.
- Commit at least once per working session so the history shows steady progress.
- **`.env`** is ignored from the first commit; a **`.env.example`** lists variable names only, never values.

15. Deployment and configuration

Frontend and backend deploy separately from the same repository, and a push to `main` redeploys both. Platforms are a suggestion; confirm AUREX's approved options before choosing.

Three free-tier hosts deploy automatically from one GitHub repository



Environment variables. Values below are placeholders; real values go only in `.env` locally (ignored by Git) and in the host dashboards.

Variable	Where	Purpose	Example
PORT	Backend	Port the server listens on (the host usually sets it)	5000
NODE_ENV	Backend	Production hides error stack traces	production
MONGO_URI	Backend	Atlas connection string	mongodb+srv://USER:PASSWORD@CLUSTER/servicelink
JWT_SECRET	Backend	Signs and verifies tokens	long random string, generated by you
JWT_EXPIRES_IN	Backend	Token lifetime	7d
BCRYPT_SALT_ROUNDS	Backend	Password hashing cost	10
ADMIN_EMAIL, ADMIN_PASSWORD	Backend, seed only	Creates the first admin account	chosen by you

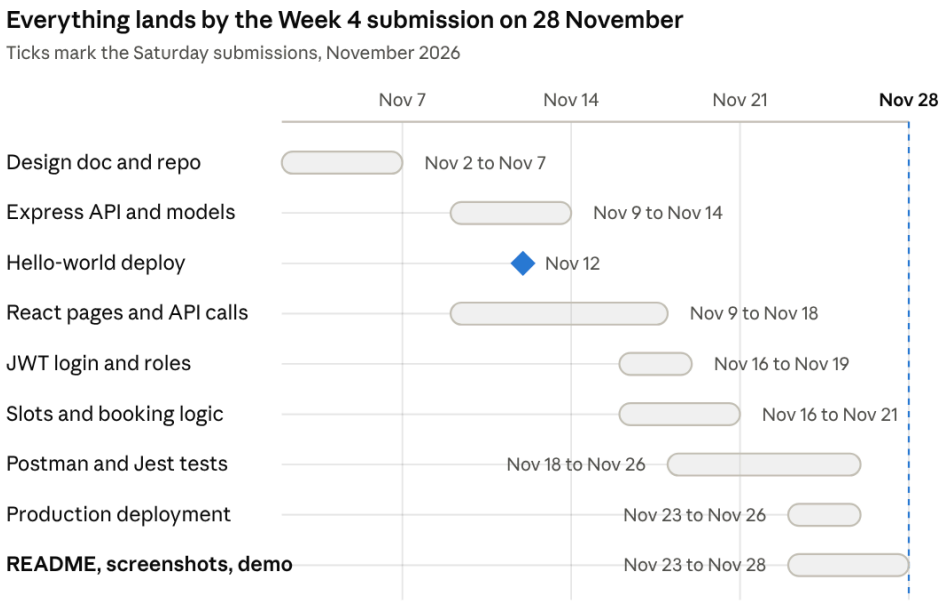
Deployment steps

1. Atlas: create a free cluster, a database user, and a network rule; put the connection string in your local **`.env`** only.
2. Render: create a Web Service from the repository with root directory `backend`, build command **`npm install`**, start command `npm start`, and the backend variables; check `/api/health`.
3. Run **`seedAdmin.js`** once against the Atlas database to create the admin.
4. Vercel: import the repository with the root directory `frontend` (Vite), set **`VITE_API_BASE_URL`**, and add a rewrite so every path serves **`index.html`**.
5. Set **`CLIENT_URL`** on Render to the Vercel address and redeploy the backend.
6. Run test case TC-28 on the live URLs and put both URLs in the README.

16. Development plan

Four weekly milestones, each ending on a Saturday submission. Dates assume Week 1 closes on Saturday 7 November 2026; shift them if your cycle started on a different day.

Week	Saturday	Focus	Done when
1	7 Nov	Planning, design, repository setup	This document, diagrams, API list, repo with README and first commits are submitted
2	14 Nov	Core build: Express, Mongoose models, React shell	Categories, providers, services and slots work end to end from React to MongoDB
3	21 Nov	Auth, roles, booking logic, testing	Booking flow with double-booking check, role-protected routes, Postman collection and test evidence
4	28 Nov	Polish, deployment, documentation, demo	Live frontend and API URLs, final README, screenshots, recorded demo



Risks and how scope is controlled

- Slot generation is the hardest logic: build it in Week 2 as a plain function with tests before wiring it to the UI.
- If Week 3 runs late, drop the Should items first (email, earnings, image upload); never drop validation, testing, or deployment.
- Deploy a hello-world version of backend and frontend in Week 2 so Week 4 is not the first time anything is hosted.
- Be able to explain every file: write a two-line note on each module as you finish it, to prepare for the final review.

17. Assumptions, constraints and scope

Assumptions

- One person builds the project in about 40 hours over four weeks (10 hours a week).
- Payments are handled offline (cash on completion), so no payment gateway is needed for the MVP.
- Each provider works from one city and serves customers who come to them or whom they visit; no travel-time calculation.
- A booking is for one service and one slot; multi-day or recurring bookings are not supported.
- Users have a modern browser and an internet connection; the app is not an installable mobile app.

Constraints

- Must use React, Node.js, Express.js, MongoDB with Mongoose, JWT, and bcrypt, as the Month 3 roadmap requires.
- Free hosting tiers: the backend may take up to a minute to wake after inactivity, and storage is limited.
- Work must be original and explainable line by line; AI tools may help but not write the whole application.
- Secrets must never appear in the repository.

Out of scope for the MVP

- Online payments, refunds, and invoices
- Real-time chat between customer and provider
- Map-based search and distance sorting
- Native mobile apps and push notifications
- Multi-language interface and recommendation engine